

LCP

Local Chat Protocol **Specification Document**

Course: Backend Development on .NET

Student: Giorgi Gogichashvili

1. Overview and Design Goals

The Local Chat Protocol (LCP) is a custom application-layer protocol designed to facilitate peer-to-peer, simplex chat communication over a Local Area Network (LAN). LCP is a hybrid protocol. It borrows its decentralized discovery mechanism from SSDP (Simple Service Discovery Protocol) to announce TCP binding details over UDP. For message framing and state transitions, LCP is inspired by the human-readable, CRLF-delimited commands of SMTP, and the strict transactional request-response model of early HTTP.

1.1 Design Goals:

- **Reliability:** Utilize TCP for guaranteed, ordered delivery of chat messages.
- **Decentralized Discovery:** Utilize UDP broadcasting to eliminate the need for a central server, allowing dynamic IP and port discovery.
- **Simplicity & Strict Turn-Taking:** Enforce a strict simplex (Stop-and-Wait) state machine to ensure only one user transmits data at a time.
- **Stability:** Gracefully handle network interruptions, deadlocks, protocol violations, and port unavailability.

2. Description of All Message Types

To ensure proper parsing and state transitions, the Local Chat Protocol (LCP) defines five distinct message types, each identified by a unique uppercase prefix.

2.1 Discovery Message (DISCOVER)

- **Sender:** Initiator
- **Receiver:** Broadcast (All nodes on the subnet listening on UDP Port 15000)
- **Purpose:** This message is the bootstrapping mechanism of the protocol. It is sent as an unreliable UDP broadcast. It contains the intended recipient's nickname, the timeout threshold (deadline), the dynamically assigned TCP port on which the Initiator has started listening, and a unique session UUID.
- **Behavior:** Nodes that receive this message but do not match the <RecipientNickname> field must silently discard the packet. The matching recipient uses the sender's IP and the <TcpPort> to initiate a TCP connection.

2.2 Handshake Request Message (HANDSHAKE_REQ)

- **Sender:** Recipient
- **Receiver:** Initiator
- **Purpose:** This is the first message transmitted over the newly established TCP connection. It serves as identity proof. By echoing back the UUID received in the initial UDP broadcast, the Recipient proves to the Initiator that it is indeed the authorized communication partner and not an arbitrary machine connecting to the open port.
- **Behavior:** This message must be sent immediately upon TCP connection establishment. If any other message type is received by the Initiator before a HANDSHAKE_REQ, the connection is immediately terminated.

2.3 Handshake Response Message (HANDSHAKE_RESP)

- **Sender:** Initiator
- **Receiver:** Recipient
- **Purpose:** This message concludes the handshaking phase. The Initiator validates that the received UUID matches its active request and verifies that the system's current UTC time has not exceeded the designated DeadlineUtc.
- **Behavior:**
 - If valid, the Initiator transmits HANDSHAKE_RESP;ACCEPT, transitioning both applications into the active simplex chat loop.
 - If invalid (expired deadline or mismatched UUID), the Initiator transmits HANDSHAKE_RESP;REJECT, terminates the TCP socket, and stops listening.

2.4 Text Message (TEXT)

- **Sender:** Bidirectional (Enforced by turn-taking rules)
- **Receiver:** Bidirectional
- **Purpose:** This message carries the actual conversation payload (the chat messages) between the two users.
- **Behavior:** To ensure simplex operation, a node can only transmit a TEXT message when it is in the WAIT_SEND state. Sending a text message instantly locks the sender's console and transitions its state to WAIT_RECV. Conversely, receiving a TEXT message unlocks the receiver's console, transitioning it to WAIT_SEND.

2.5 Connection Close Request Message (CLOSE)

- **Sender:** Bidirectional
- **Receiver:** Bidirectional

- **Purpose:** This message initiates a graceful shutdown of the TCP session. It signals to the remote peer that the sender is ending the session, allowing both sides to cleanly flush buffers, close streams, and release socket resources back to the operating system.
- **Behavior:** Upon receiving a CLOSE message, the recipient terminates its message loop, displays the attached shutdown reason, and shuts down its application.

3. Message Format Definitions

All LCP messages follow a strict prefix-based structure. Fields are delimited by a semicolon (;).

3.1 UDP Discovery Message

Sent by the Initiator to dynamically discover the Recipient and announce connection parameters.

Field Index	Name	Type	Description
0	Prefix	String	Exactly `DISCOVER`
1	Nickname	String	The target Recipient's nickname
2	DeadlineUtc	String	ISO 8601 formatted UTC timestamp defining when the connection offer expires
3	TcpPort	Integer	The dynamically allocated TCP port the Initiator is listening on
4	UUID	String	A randomly generated UUIDv4 for session validation

Example:

DISCOVER;Giorgi;2026-06-14T15:00:00.0000000Z;49743;f81d4fae-7dec-11d0-a765-00a0c91e6bf6

3.2 TCP Handshaking Request

Sent by the Recipient immediately after establishing the TCP connection.

Field Index	Name	Type	Description
0	Prefix	String	Exactly `HANDSHAKE_REQ`
1	UUID	String	The exact UUID captured from the UDP broadcast

Example:

HANDSHAKE_REQ;f81d4fae-7dec-11d0-a765-00a0c91e6bf6

3.3 TCP Handshaking Response

Sent by the Initiator to validate the Recipient's identity and enforce the timeout deadline.

Field Index	Name	Type	Description
0	Prefix	String	Exactly `HANDSHAKE_RESP`
1	Status	String	`ACCEPT` if validation succeeds, `REJECT` if validation fails
2	Message	String	Human-readable reason for the status.

Success example:

HANDSHAKE_RESP;ACCEPT;Handshake verified.

Fail example:

HANDSHAKE_RESP;REJECT;Deadline expired.

3.4 TCP Text Message

Used for active payload transmission. Note: To allow semicolons in user input, the payload is read dynamically bypassing standard delimiter splitting.

Field Index	Name	Type	Description
0	Prefix	String	Exactly `TEXT`
1	Payload	String	The user's chat message (may contain spaces and semicolons)

Example:

TEXT;Hello, how are you doing today?

3.5 TCP Connection Close Request

Sent by either peer to request a graceful termination of the connection.

Field Index	Name	Type	Description
0	Prefix	String	Exactly `CLOSE`
1	Reason	String	Optional human-readable reason for disconnect

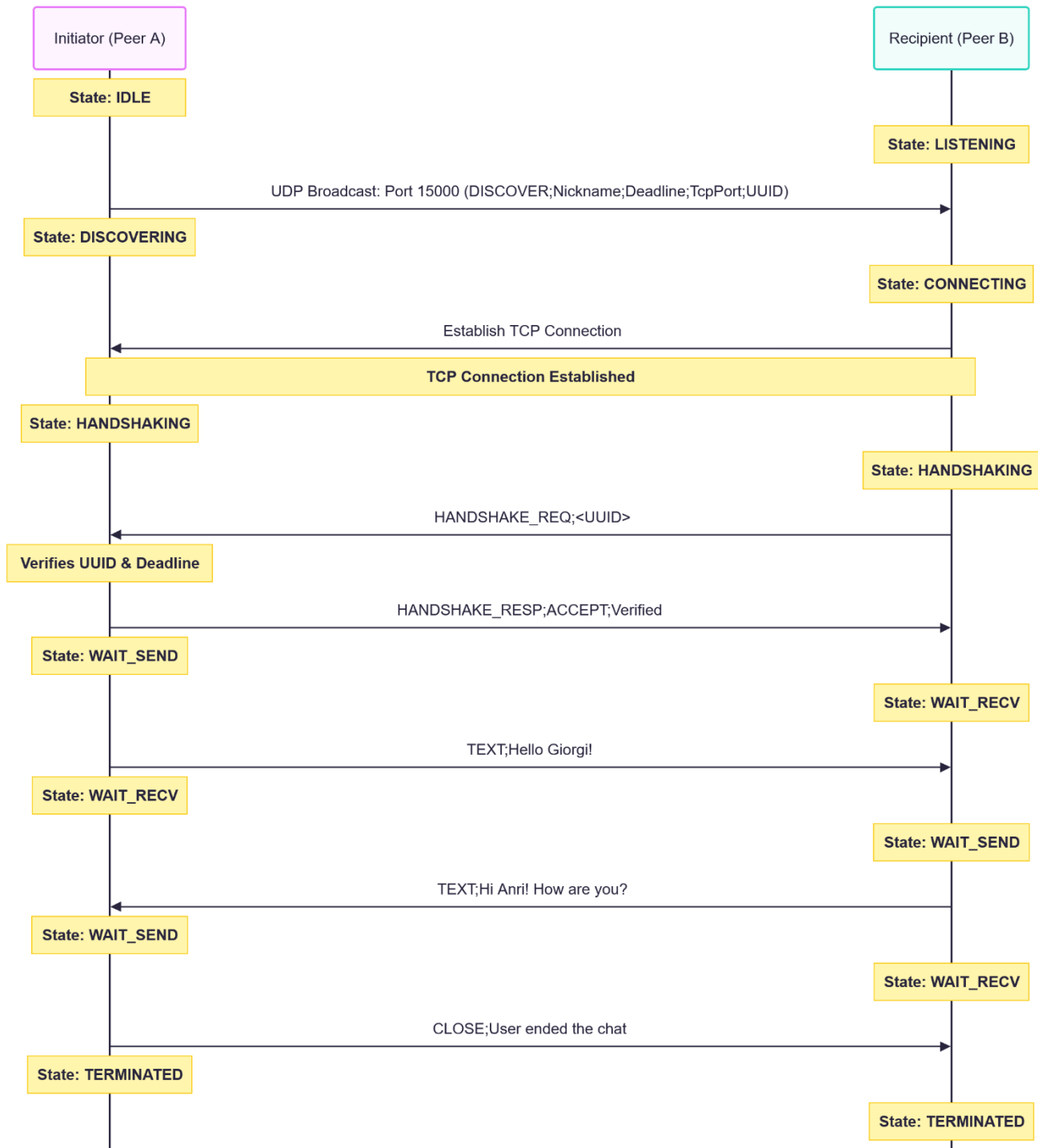
Example:

CLOSE;User ended the chat

4. Communication Workflow and State Transitions

LCP enforces a strict State Machine to guarantee simplex communication.

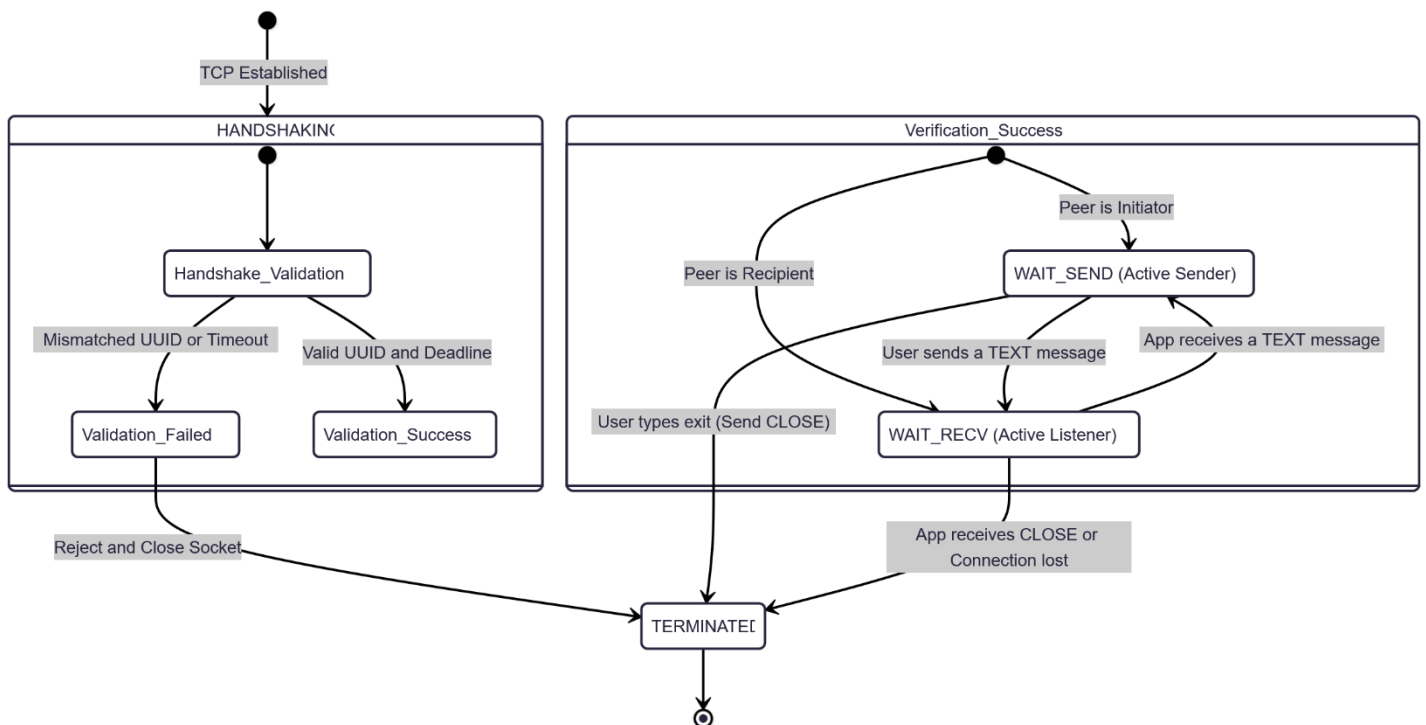
4.1 Sequence Diagram



4.2 Simplex State Toggle Rule

Once the Handshake is successfully accepted:

- 1 The Initiator enters the **WAIT_SEND** state (granted the first transmission token).
- 2 The Recipient enters the **WAIT_RECV** state.
- 3 Upon transmitting a **TEXT** message, the sender automatically transitions to **WAIT_RECV**.
- 4 Upon receiving a **TEXT** message, the receiver automatically transitions to **WAIT_SEND**.



While the sequence diagram outlines the network messaging over time, the following state machine diagram illustrates the internal execution states of a single peer node. This diagram shows the strict simplex toggle logic, emphasizing how both roles (Initiator and Recipient) share the same underlying state loop but enter it from opposite phases upon successful handshake verification.

5. Assumptions and Limitations

- **Network environment:** Both peers must reside on the same Local Area Network (LAN) or Virtual Local Area Network (VLAN) that permits UDP broadcasting.
- **Encoding:** All messages are transmitted as UTF-8 encoded text.
- **Framing:** The protocol relies on Carriage Return + Line Feed (\r\n) to define the end of a message boundary over the TCP stream.
- **Simplex mode:** Full-duplex communication is strictly prohibited. A peer in the WAIT_RECV state cannot transmit payload data until it receives a message and transitions to WAIT_SEND.
- **Broadcast domain limitations (No routing):** Because LCP relies on UDP broadcasting (255.255.255.255), the discovery phase is strictly limited to a single broadcast domain (subnet). UDP broadcast packets are discarded by routers and layer-3 switches, meaning users residing on different subnets or connecting via a VPN/WAN cannot discover each other.
- **Lack of encryption:** The protocol does not implement cryptographic wrapping (such as TLS/SSL). All payloads, including nicknames, UUIDs, and chat messages are transmitted across the local network in plaintext. The protocol assumes a trusted local network environment and is vulnerable to packet-sniffing and man-in-the-middle attacks.
- **Single-Session constraint:** LCP is strictly a peer-to-peer, single-session protocol. The Initiator's TcpListener only accepts a single incoming connection. Once a handshake is validated, no other users can connect, and there is no native support for multi-user chat rooms or group messaging.
- **No message persistence or history:** LCP is a stateless session protocol. Messages are streamed directly to the terminal interface and do not persist to a database, file, or local cache. Once a socket is closed, the entire chat history for that session is permanently lost.
- **Operating System and Firewall dependency:** The protocol assumes that local operating system firewalls allow UDP traffic on port 15000 and inbound/outbound TCP traffic on the dynamically negotiated port. If strict security profiles (such as public network profiles on Windows) block these ports, the protocol will fail to bind or connect.

6. Error Handling and Edge Cases

The protocol dictates specific behaviors for anomalous conditions and edge cases:

1. **Port Exhaustion / Collision:** If the Initiator's preferred TCP listening port is occupied, the application dynamically requests an available ephemeral port (Port 0) from the OS. This port is then correctly injected into the UDP discovery packet, ensuring the Recipient connects flawlessly.
2. **Network Timeout:** If the Recipient matches the broadcast but connects after the DeadlineUtc timestamp, the Initiator intercepts the HANDSHAKE_REQ, issues a HANDSHAKE_RESP;REJECT, and terminates the socket.
3. **Unexpected Socket Drop (RST):** If a peer violently crashes (generating a TCP Exception), the active StreamReader will trap the exception, simulate a CLOSE;Connection dropped event, and exit the chat loop without crashing the application.
4. **Protocol Violation / Data Corruption:** If a peer receives data that lacks a valid TEXT; or CLOSE; prefix during the chat phase, it transmits a CLOSE;Protocol violation command and immediately tears down the TCP session.
5. **Two Initiators looking for the same Recipient:** If two separate Initiators on the same local network simultaneously broadcast a discovery message looking for the exact same Recipient, because the Recipient application processes incoming UDP packets sequentially, It will process the first broadcast it receives, initiate a TCP connection to that first Initiator, and transition its state to HANDSHAKING. Once connected, the Recipient stops listening to UDP port 15000. The second Initiator will receive no connection attempt and will safely time out once its DeadlineUtc expires, avoiding any double-connection conflicts.
6. **Empty message transmission:** To prevent the transmission of empty and null packets over the network, both applications must validate user input locally. If the input is null, empty, or consists purely of whitespace, the application suppresses the transmission, refuses to send the TEXT; packet, and remains in the WAIT_SEND state, forcing the user to type a valid message before the simplex token is passed.